

Comprehensive Documentation

A. Overview

1. Purpose

The primary objective of this project is to build a Clothing Assistant chatbot. This system is designed to perform two main functions:

- **Store Policy & Customer Help Desk (Part A):** The chatbot answers customer questions regarding store policies (e.g., return policies, discounts).
- **AI Stylist (Part B):** The agent suggests clothes based on user queries (e.g., 'suggest me an outfit for a graduation party') and leverages customer context (etc). A key goal is preventing hallucinations by suggesting products only from the company catalogue.

2. Architecture Summary

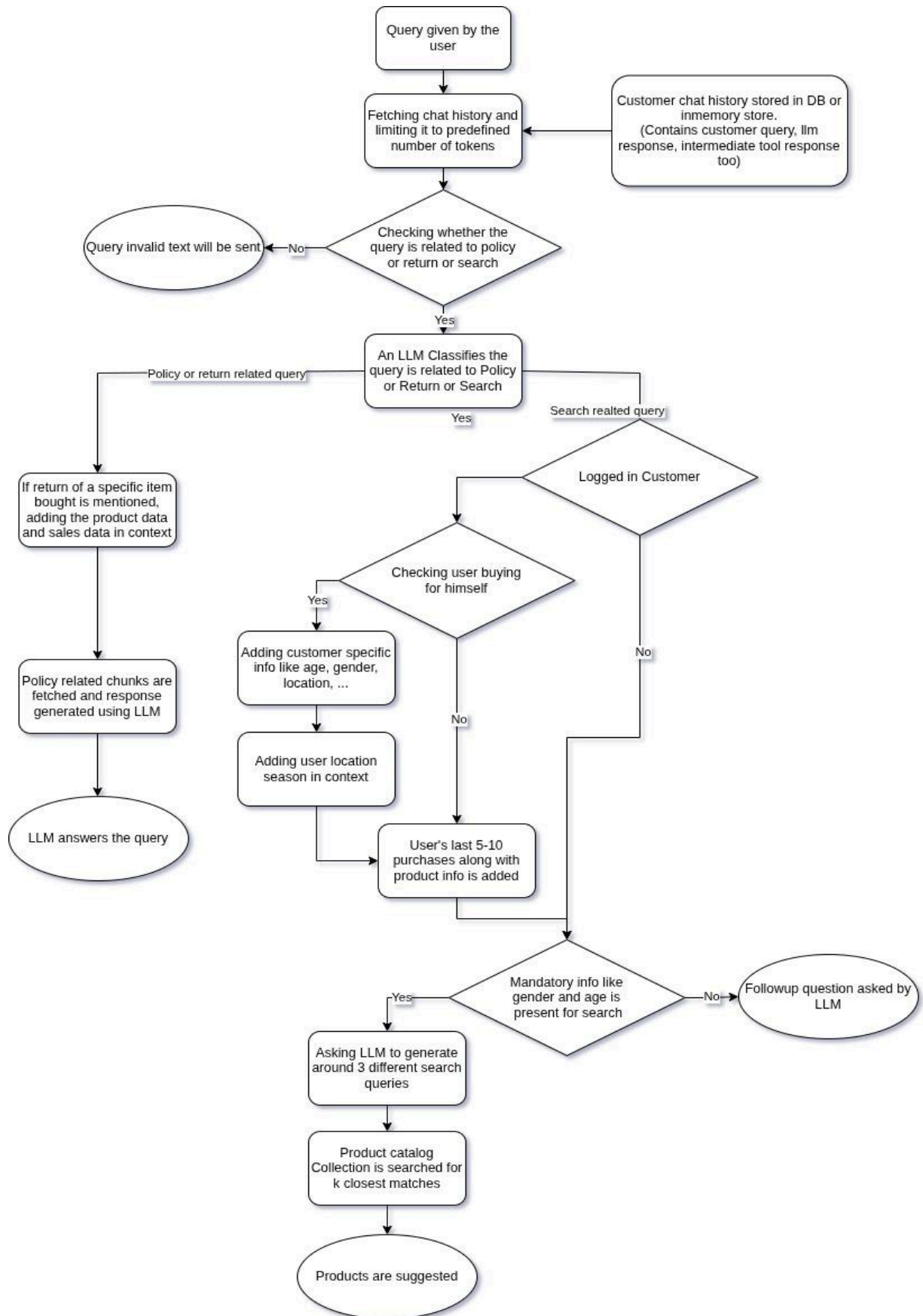
The system is implemented as a Retrieval-Augmented Generation (RAG) pipeline, which splits the functionality into a Policy LLM and an AI Stylist (Product RAG).

High-Level System Diagram and Data Flow:

The overall flow for product search is highly conditional and involves multiple LLM interactions:

- **Input:** Query given by the user.
- **Context Building & Classification:** Fetch chat history. An LLM classifies the query as related to Policy, Return, or Search.
- **Policy/Return Path:** If classified as Policy/Return, policy- related chunks are retrieved from the Vector DB, potentially supplemented by product/sales data if a specific return is requested, before the LLM generates the answer.
- **Product Search Path (AI Stylist):**

- a. **Context Addition:** Customer-specific information (age, gender, location, purchase history) is added to the context.
- b. **Mandatory Info Check:** If mandatory info (like gender and age) is missing for the search, the LLM asks a follow-up question.
- c. **Query Refinement:** The user query and customer context are passed to a dedicated LLM (refine_prompt_balanced) to rewrite the query into a refined, detailed version.
- d. **Retrieval:** The refined query is converted into numeric embeddings using the BGE model. This embedding is used to compute cosine similarity against the product catalog stored in the Vector DB (VDB).
- e. **Top-K Retrieval:** The top 12 most relevant products are retrieved.
- f. **Reranking/Output:** The top retrieved products and the refined query are passed to another LLM (rerank_prompt_balanced) to identify the top 3 most suitable products in a structured format.
- **Output:** Products are suggested or the policy answer is returned.



3. Deployed Components:

The core components are driven by the RAG architecture:

- **Frontend/UI:** A basic frontend UI for the ChatBot has been implemented, specifically using Streamlit for the Policy RAG component.
- **Backend Model (LLM):** The system uses a large language model, which is Gemini free API. LLMs are used for query classification, refinement, reranking, and generation.
- **Backend Model (Embedders):**
 - For Policy RAG, the BAAI/bge-base-en-v1.5 model was chosen for production due to its superior performance.
 - For the Product RAG, the same BGE model is used to convert product descriptions and refined queries into numeric embeddings.
 - Vector Database (VDB): The system uses FAISS and ChromaDB for product and policy embeddings respectively
 - Context/Memory Store: Customer session and memory management is maintained in a class instance.

B. Technical Documentation

1. Environment Setup:

The environment requires specialized models and libraries for RAG and data handling:

- **Python Libraries:** flask, requests, streamlit, PyMuPDF, sentence-transformers, scikit-learn, groq, faiss-cpu, numpy, pandas, chromadb, langchain-core, langchain-community, langchain-google-genai, langchain-groq, langchain-classic, pydantic, unicorn.

- **Hardware:** BGE models offer good performance on commodity GPUs as well as CPUs, making them suitable for scalable retrieval.
- **Vector Database:** Integration with ChromaDB is used for policy embeddings.

2. Data Pipeline

Summarize preprocessing, feature extraction, and dataset sources. Include licensing info and where data lives.

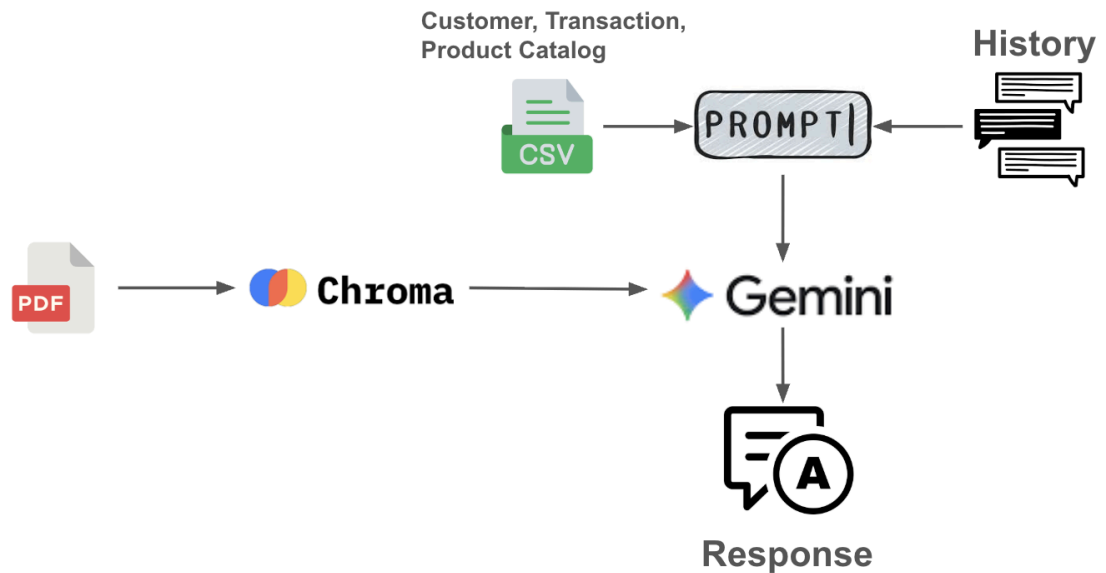
- **Policy RAG:**
 - We created the dataset (policy doc) by ourselves and with the help of LLMs. We also referred to other online policy docs for reference.
 - The policy doc segregated in sub-heading level and each segregated part.
 - The segregated policies is now chunked using a chunk size of 120 each chunk is also appended with the policy's subheading and keywords present in the chunk.
 - For each chunk 5-7 keywords are generated using `llama-3.1-8b-instant`
 - Now the preprocessed text is embedded using BGE sentence-transformer and stored in chroma DB.
- **Product RAG:**
 - The dataset we used for the product catalog is taken from a Kaggle competition. [\[Link\]](#)
 - The dataset contains some unwanted columns like style code, category code along with its descriptors. But these codes are not needed for our project. We removed these columns.
 - After making sure only necessary columns are present, each row's data is combined and embedded using the BGE model and the embedding is saved in a parquet file for future use.

3. Model Architecture

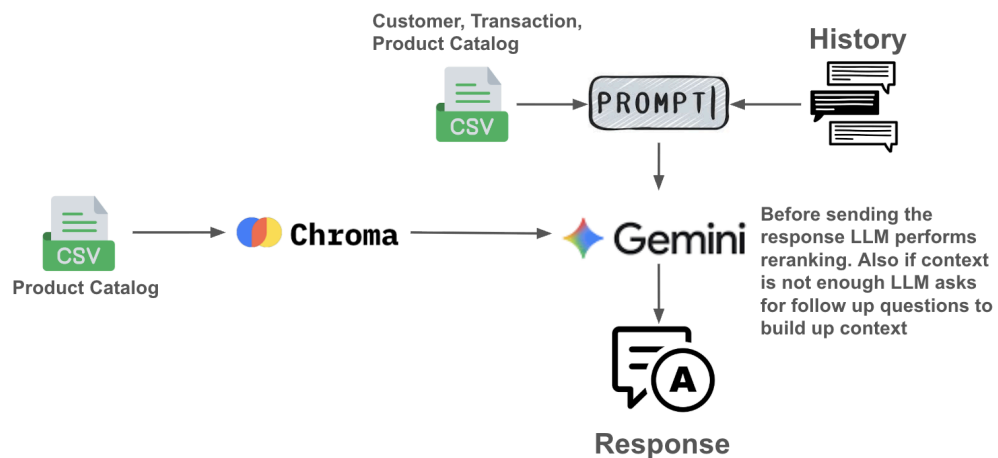
The core models involve Retrieval-Augmented Generation (RAG) utilizing a multi-stage approach for policy retrieval or product recommendation.

- **Model Architecture:** RAG Pipeline with separate flows for Policy and Product queries.
- **Embedder Model:** BAAI/bge-base-en-v1.5 is the preferred model for Policy RAG. BAAI/bge-small-en model is used for product embedding.
- **LLM:** Gemini 2.5 flash

Policy RAG



Product RAG



4. Training Summary - Hyperparameter tuning, Embedding Model selection, Prompt Engineering:

- **Policy RAG Tuning:**

- We prepared 5 policy related queries for the hyperparameter tuning and ran it through our pipeline.
- Tested two models (BAAI/bge-base-en-v1.5 and all-MiniLM-L6-v2) and two retrieval/chunking combinations (e.g., Chunk size 700/Overlap 70 and 300/30). BAAI/bge-base-en-v1.5 performed better, achieving an average manual score of 4.8 out of 5.

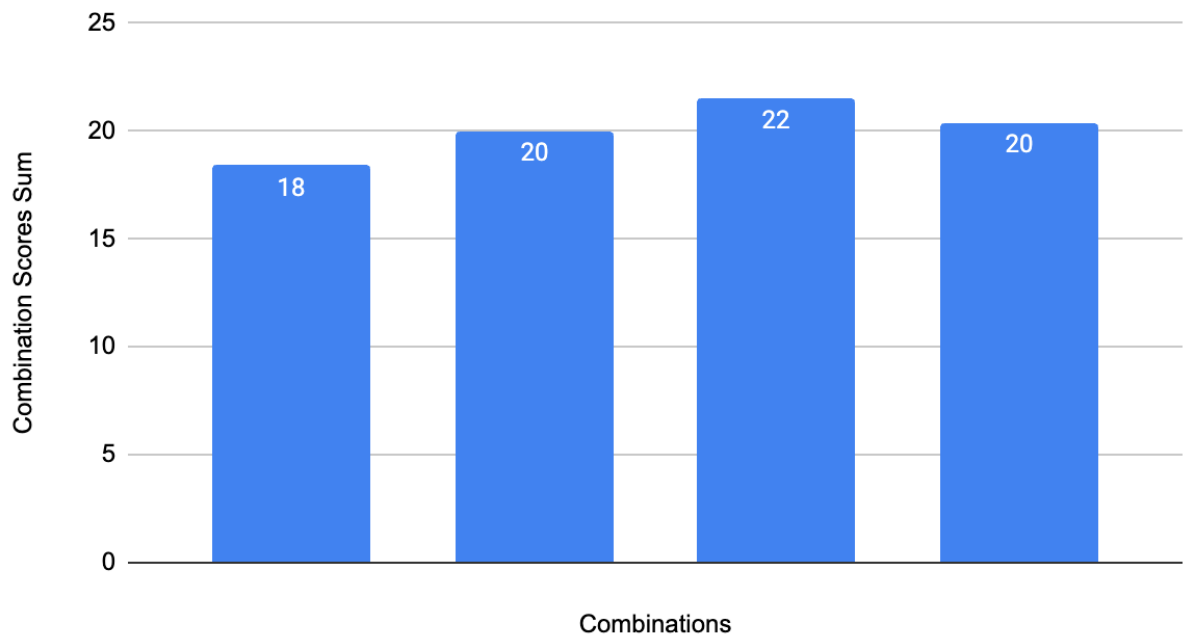
Query	combination 1	combination 2
1	5	3
2	4.5	4.5
3	5	4
4	5	3
5	4.5	0
Average	4.8	2.9

- **Product RAG Tuning:**

- We prepared 10 product related search queries, added customer context, transaction history and chat history too.
- Four combinations of Temperature, Top-K, Refine Prompt, and Rerank Prompt were tested. Scoring was performed manually, with an average of the three product suggestions used as the query score. Combination 3 showed the best performance.

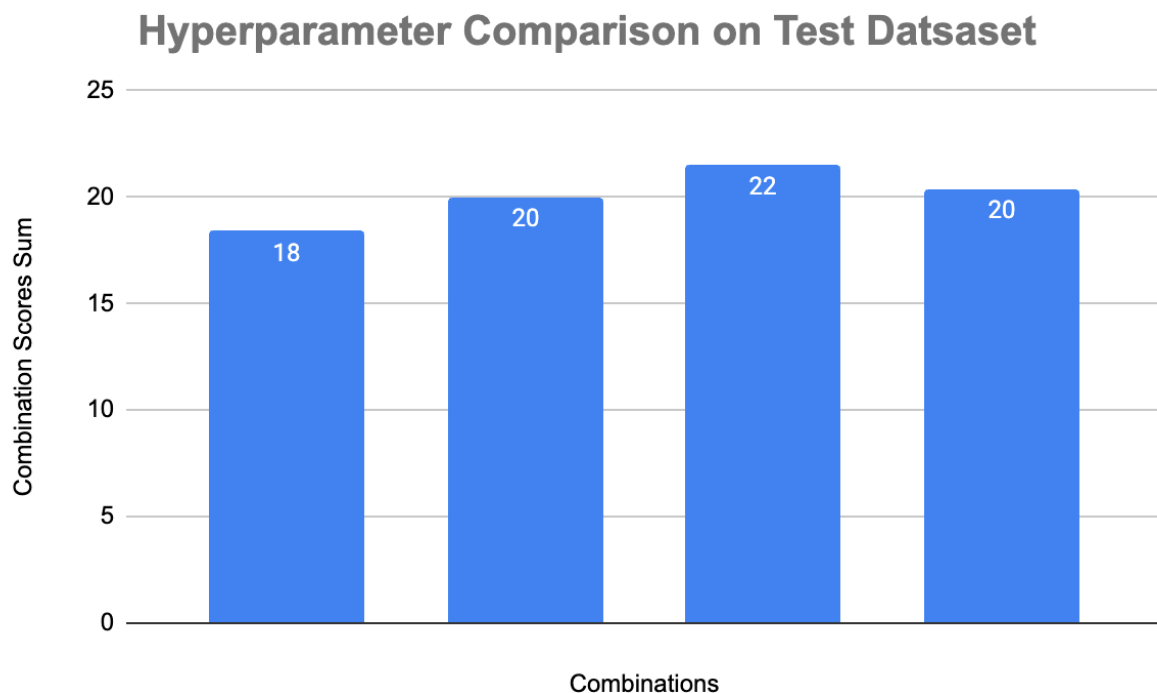
- **The LLM reranker uses specific weightage logic:** 80% for the current user query meaning, 15% for customer context (age, gender, location, preferences), and 5% for purchase history (style consistency).
- **Final Hyperparameters (Product RAG - Combination 3):** This configuration consistently outperformed others.
 - LLM Temperature: 0.5 (Enables controlled randomness/creativity).
 - Top-K (Retrieval Limit): 12 (Number of products fetched before reranking).
 - Refine Prompt: refine_prompt_balanced.
 - Rerank Prompt: rerank_prompt_balanced.

Hyperparameter Comparison on Test Datsaset

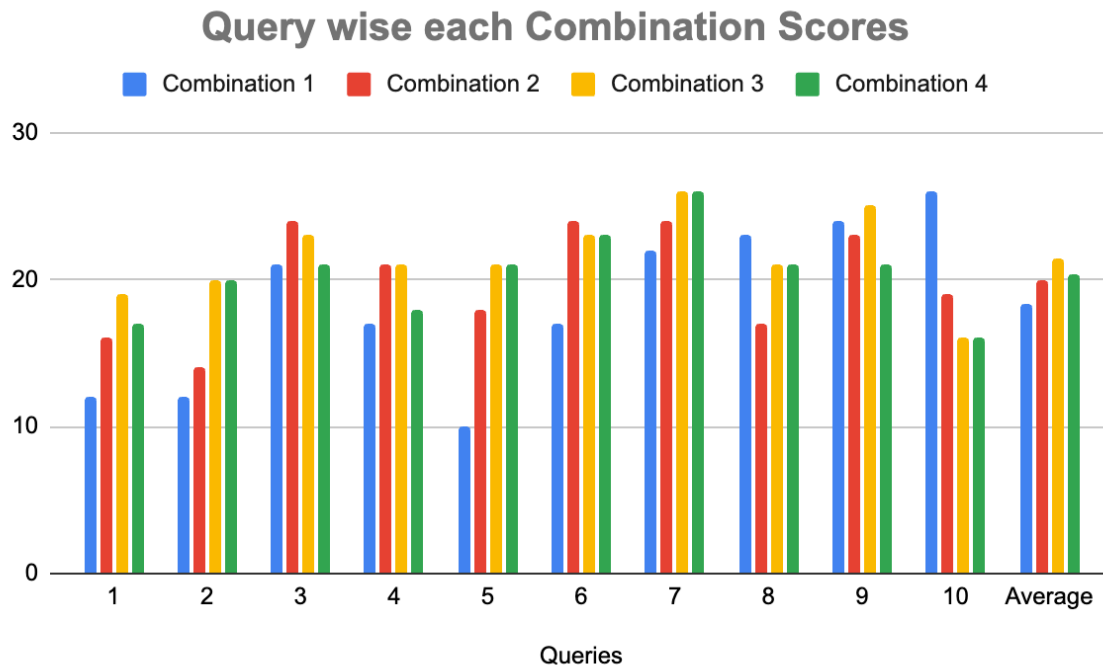


5. Evaluation Summary

- Policy RAG:
 - While hyperparameter optimization we found the initial chunking strategy (where policies were not segregated based on sub-headings) didn't work well. So, we segregated based on sub-headings before chunking.
 - Unlike hyperparameter optimization, we used the RAGAS scoring framework instead of manual scoring.
 - A set of 10 policy queries were run through our pipeline and we scored the results.
 - Refer to this [sheet](#) for more information.
- Product RAG:
 - Similar to the training query set, we prepared 10 product search queries along with customer context, transaction history and chat history.
 - Also, the products on which the testing is done were not used while hyperparameter tuning.



- A similar result compared to training scores were obtained where combination 3 hyperparameter combinations worked the best.
- Refer to this [sheet](#) for more detailed information on the query, customer context, transaction history and chat history used.



● Observed Limitations & Errors:

- **Filtering Issues:** The system struggled to apply multiple filters accurately (e.g., results for "returnable blue t-shirt" included non-returnable items).
- **Category Confusion:** Embeddings for similar product types like "shirt" and "t-shirt" were too close, leading to mixed results.
- **Sparse Context:** When customer context or query detail is minimal, the search results can be random.
- **Memory/Latency:** Large context increases the token count, which increases the latency of LLM calls.
- **Missing Features:** The current version cannot omit already purchased items or search based on user-uploaded images.

6. Inference Pipeline

The inference pipeline is driven by context-aware steps.

- **Inference Steps (Policy Query):**

- If the initial LLM call classifies the query as a policy related question this flow is followed.
- Along with the query customer context is appended.
- Now, the query along with other meta is embedded and essential chunks to answer the query are fetched using the embedding of the query.
- A final LLM call is made to answer the query.

- **Inference Steps (Product Query):**

- **Context Assembly:** Customer context (customer details, transaction history, etc.) is gathered using pandas from CSVs. If mandatory information age, gender is not available, the same is requested before proceeding further.
- **Query Refinement:** The user query and assembled customer context are input into the refine_prompt LLM to generate a semantically rich query.
- **Vector Search:** The refined query is embedded using the BGE model. Cosine similarity is computed against all product embeddings in the VDB. The top 12 most relevant products are retrieved.
- **Reranking/Selection:** The top retrieved items and the refined query are passed to the second LLM (rerank_prompt_balanced). This LLM applies the 80/15/5 (current user query/customer context/purchase history) weightage logic to select the final top 3 products.
- **Output:** The final top 3 products are presented in a structured format.
- **Example of Product Output** (Combination 3, Query 1):
 - Product 1:
Name: Roll Up Tee, Type: T-shirt, Color: Light Red, Price: 664, Discount: 24%, Return Type: No return, SKU: 541706015.

7. Deployment Details

1. Overview

For the deployment of our full-stack application, we initially attempted to use Render due to its simple workflow and free tier support. While the frontend deployment was successful, the backend failed because of high memory requirements.

To overcome this, we migrated our deployment to **Google Cloud Platform (GCP)** using a **Virtual Machine (VM)**.

Both the **frontend and backend** are now successfully deployed and running on the GCP VM.

2. Frontend Deployment (Render → GCP)

2.1 Initial Deployment on Render

- The frontend was deployed using Render's static site hosting.
- URL: <https://group-5-ds-ai-lab-project.onrender.com/>
- **Status:**
 - Deployment successful
 - Fully functional
 - Responsive and publicly accessible
- **Included components:**
 - Product search interface
 - Dashboard
 - Overall UI

Render handled the frontend smoothly because it required minimal computation and memory.

2.2 Current Deployment on GCP VM

- The frontend has now also been deployed on the **GCP VM** along with the backend.
- Running on the same VM instance ensures better integration and zero CORS issues.

3. Backend Deployment

3.1 Attempt on Render (Failed)

Backend URL:

<https://group-5-ds-ai-lab-project-1.onrender.com/>

Status: **Failed to run**

Reasons for Failure:

- Our backend includes:
 - A large machine learning model
 - Embedding vectors
 - Product dataset
 - Additional mapping and preprocessing files
- These components require **2–4 GB+ RAM**, but Render's free tier offers **only ~512 MB**.
- The Render container repeatedly crashed due to:
 - Out-of-Memory (OOM) errors
 - Backend failing to boot

4. Successful Deployment on Google Cloud Platform (GCP)

4.1 VM Configuration

We deployed the entire application on a single GCP VM instance with:

- **8 GB RAM**
- **~16 GB disk**
- Sufficient compute resources to load:
 - ML model
 - Embedding vectors
 - Dataset
 - API services

This configuration resolved all previous memory-related issues.

4.2 Deployment Status

- **Frontend running successfully on GCP VM**
- **Backend running successfully on GCP VM**
- Model, embeddings, and dataset load without failure
- The VM is stopped when not in use to save credits

4.3 Note on Ephemeral IP Address

Since we are using an **ephemeral external IP**, every time we stop and restart the VM:

- The **public IP changes**
- We must update the frontend/backend URLs after each restart

To avoid this, upgrading to a **static external IP** is recommended, but it costs additional credits.

5. Conclusion

- **Frontend Deployment:** Successfully deployed first on Render, and now fully running on the GCP VM.
- **Backend Deployment:** Render deployment failure due to memory limitations was resolved by migrating to an 8 GB RAM GCP VM.
- **Current Deployment:** Both frontend and backend run smoothly on the GCP VM.
- **Limitation:** Due to limited GCP credits, we stop the VM when not in use. As a result, the public IP is **ephemeral** and changes after each restart.

8. System Design Considerations

The system uses a modular RAG design with two primary functional pipelines (Policy and Product) managed by classification logic.

- **Data Flow and Modularity:** The architecture separates concerns, using an LLM for classification before directing the query to either the Policy RAG (based on current query and chat history) or the Product RAG (which uses query refinement, vector search, and reranking).
- **RAG Interaction:**
 - The retriever retrieves embeddings (using cosine similarity) from the Vector DB (VDB).
- **Memory Management:** Conversational memory will be maintained. To mitigate token limits and high latency caused by growing chat history, conversation summarizing

techniques (like ConversationSummaryMemory) are planned, potentially using Redis for persistent session storage.

9. Error Handling & Monitoring

- **Query Validation:** An initial check is performed to ensure the query is related to the chatbot's tasks (policy, return, or search); otherwise, an invalid text response is sent.
- **Context Management:** If mandatory context information (like age or gender) is missing, the system automatically sends a follow-up question to the user. The RAG pipeline was observed to be 100% effective in detecting missing age or gender information during testing.
- **Latency:** A known limitation is that latency increases as the chat history grows in token count.
- **Product Irrelevance:** During product retrieval, if highly relevant results are scarce, the system may return non-matching products to meet the requested output quota (e.g., if only one product matches a filter, the remaining two may be wrong).

10. Reproducibility Checklist

To replicate the core RAG results:

- **Source Data:** The product catalog data is derived from the H&M Personalized Fashion Recommendations Kaggle competition.
- **LLM/Embedders:**
 - Use an LLM (such as the Gemini API) for the generative components (refinement, reranking, answer generation).
 - Use the BAAI/bge-base-en-v1.5 model for embedding documents and queries.
- **Configuration Files/Hyperparameters:** The system configuration should use the optimal settings determined in Milestone 5:

- **Temperature:** 0.5
- **Top-K (Retrieval):** 12
- **Prompts:** refine_prompt_balanced and rerank_prompt_balanced.
- **Data Preparation:** Ensure that unwanted product columns are removed, and product embeddings are normalized.

Since, the best temperature for our product RAG was 0.5 the product suggestions might vary due to differences in search queries. This can be set to 0 for reproducible results. When temperature is set to 0, the policy RAG answers will also be reproduced.

C. User Documentation (for Non-Technical Users)

1. App Overview:

- The AI Fashion Chatbot acts as your Clothing Assistant, designed to handle both customer support inquiries and personalized fashion recommendations.

2. Key Use Cases:

- **Store Policy & Customer Help Desk (Part A):** Ask questions regarding general store rules, such as return policies, upcoming discounts, or refund timelines.
- **AI Stylist (Part B):** Get personalized outfit suggestions based on your current request (e.g., 'suggest me an outfit for a graduation party'). The agent uses your context (age, gender, purchase history) to refine suggestions. The system is designed to only suggest products available in the company catalogue to prevent irrelevant suggestions.

3. Input Description:

Users interact with the system via a ChatBot UI (implemented using Streamlit).

- **Action: Login**

- You must type your customer ID using the format: cust_id: {customer_id}.
(Valid sample IDs are 1, 3, 5) in the chat UI.

- **Action: Logout**

- You must type logout in the chat UI.

4. Policy Query:

- Ask a natural language question about store rules (e.g., "What is the refund condition?").

5. Product Query:

- Ask for a recommendation (e.g., "Need a blue jean") or a combination of style and context (e.g., "I need a shirt which goes well with cream pants").
- If you are logged out, or if mandatory information (like age and gender) is missing for a product search, the system will pause and ask you a follow-up question before proceeding.

6. Output Description:

The Chatbot's response varies based on the query type:

- **Policy Response:** The system provides a detailed, conversational answer that is grounded directly in the official policy documents.
 - **Example:** For "what is the return policy," the output includes eligibility, return conditions, non-returnable categories, and the return process.

- **Product Recommendation (AI Stylist):** The system consistently outputs the top 3 most suitable products in a structured format.
 - Each recommended product will be listed, typically including its: Name, Type, Color, Price (INR), Discount (%), Return Type, and SKU. Product images will be shown along with the text description.

Step-by-Step Instructions:

- Create a python virtual env : `python3 -m venv .venv`
- Activate the virtual env : `source ./venv/bin/activate`
- Install all requirements given in requirements.txt : `pip -r requirements.txt`
- Run streamlit : `streamlit run code/chat_ui.py`
- Run flask backend : `code/python app.py`

Now the UI will be accessible on <http://localhost:8501/>, the port might vary. Please refer to the terminal where streamlit is started.

Screenshots:

Policy related question:



AI Fashion Chatbot

Ask your question:

What is the return policy of your store?

Ask

AI Response: At Clothing Retail, you may return or exchange items purchased within the specified return/exchange period mentioned on the product page.

Return Process:

1. Log in to your Clothing Retail account.
2. Go to "My Orders" and click on Return/Exchange for the respective item.
3. Choose Return or Exchange, specify the reason, and upload supporting images (if damaged).
4. Select your preferred refund mode (original payment method or Clothing Retail Wallet).
5. A pickup will be scheduled within 4–7 days or instructions for self-shipping will be provided.

Eligibility:

- If there's a size or fit issue after accepting the product, you can raise a return or exchange request through the standard process, subject to policy and stock availability.
- If a product is found wrong, damaged, or missing during Open Box Delivery, you will be eligible for a refund only, not replacement or exchange.

Product Suggestions:



AI Fashion Chatbot

Ask your question:

I need formal shirts for office

Ask



Recommended Styles



Product

CA Gustavsberg TVP

Type: Shirt

Color: White

Price: 736 INR

Discount: 20%

Return Type: 15 days return

SKU No: 395127001

Add to Cart



Product

CA Gustavsberg TVP

Type: Shirt

Color: Light Blue

Price: 1805 INR

Discount: 8%

Return Type: 7 days return

SKU No: 395127011

Add to Cart



Product

TD check big Gingham/OPEN

Type: Shirt

Color: Dark Blue

Price: 580 INR

Discount: 5%

Return Type: 15 days return

SKU No: 551910005

Add to Cart

D. API Documentation

- Instead of using streamlit UI, the RAG pipeline can be used using /query endpoint made available using flask.
- The method used should be POST.
- Example Curl request: `curl -X POST http://localhost:5000/query -H "Content-Type: application/json" -d '{"query": "your test question here"}'`
- Input format: `{"query": "I need a blue jean"}`
- Output format: `{"response": "llm_response(may contain policy related answer, product suggestion, etc)", "images": ["paths of images of suggested products"]}`

E. Licensing & Dataset References

- **Code License:**

This project is released under the MIT License.

Copyright <2025> <DSAI-LAB-PROJECT-GROUP-5>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT

HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

- **Dataset Licenses:**

- **H&M Personalized Fashion Recommendations Dataset:**

License: Provided exclusively for non-commercial purposes and academic research. Commercial use is strictly prohibited.

Reference:

<https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations/data>

- **Store Policy Corpus:**

Synthesized from public policy documents of major retailers (H&M, Marks & Spencer, Levis) for academic reference only.

Model sources and citations:

- **BGE / E5 Embedding Models:**

Source: Microsoft Intfloat, Sentence Transformers

Citation:

- E5: <https://huggingface.co/intfloat/e5-base>
- Sentence Transformers: <https://www.sbert.net/>
- Paper: "Embedding Models from Instructions: E5" (<https://arxiv.org/abs/2212.03533>)

- **FashionSigLIP, DINOv2, FashionCLIP:**

Source:

- FashionSigLIP: [Insert official repository/citation if used]
- DINOv2: <https://github.com/facebookresearch/dinov2>
- FashionCLIP: <https://github.com/yahoo/FashionCLIP>

- **Gemini Models (Google Gemini via API):**

Source: Google

- The Gemini LLM (Large Language Model) was accessed via official Google APIs for text generation, retrieval, and conversational dialog tasks.
- API Documentation: <https://ai.google.dev/gemini-api/docs>
- Usage complies with Google's Terms of Service for cloud AI APIs.

Citation:

- "Gemini: Google's Multimodal Large Language Model" (Official API/Documentation): <https://ai.google.dev/gemini-api/docs/overview>

F. Future Work / Maintenance Notes

- **Possible Extensions:**

Several opportunities exist to enhance the chatbot's capabilities, particularly focusing on multimodal features and advanced context management:

- **Incorporating Multimodal Data:** The current system relies on text embeddings alone. A key next step is including image data into the embedding pipeline to increase embedding quality and allow for visual matching.
 - The preferred embedding model for this multimodal capability is FashionSigLIP, due to its ability to balance fine-grained fashion understanding with text-image alignment.
 - This would enable features like allowing users to upload images and search based on them (e.g., finding similar or matching clothes).
- **Advanced Context and Personalization:**
 - Implement memory and token optimized techniques, such as conversation summarizing techniques (e.g., ConversationSummaryMemory), to manage long chat histories when the number of tokens exceeds a threshold.
 - Use of persistent storage mechanism for maintaining the chat history.
 - A customer classification model can be trained to show suggestions based on the purchases of other customers who fall into the same classification.
 - A quiz can be designed initially to capture customer preferences and serve as context.
- **Model Improvement and Fine-Tuning:**
 - Embedding models could be fine-tuned using various images and their text descriptors to increase embedding quality.
 - To improve the fashion compatibility logic, the embedding model could be trained to place matching clothes closer and non-matching pairs farther, potentially achieved using loss functions like triplet loss.

- For accurate search results, metadata search using deterministic methods can be coupled with vector embedding search.

- **Known Limitations:**

The system, while functional, faces constraints primarily related to data coverage, latency, and the scope of text-only embedding:

- **Latency and Performance:** When the chat history grows large, the increasing token count significantly raises the latency of LLM calls.
- **Product Filtering Coverage:**
 - The current implementation does not allow for the omission of already purchased or disliked items.
 - When multiple filters are applied (e.g., "returnable blue t-shirt"), the vector search results still contain non-returnable products.
 - There is confusion between common categories like shirt and t-shirt because their embeddings appear too close in the vector space.
- **Sparse Results Handling:** If the customer search query is highly specific and the product catalog contains only one truly relevant product, the system may return two non-matching products to fulfill the quota of three suggestions. Also, when not much customer context is available, search results can be highly random.
- **Multimodal Deficiency:** Using text embeddings alone means the system cannot find clothes based on matching visual patterns.
- **Context Misalignment:** If a male customer asks for a "frock," the chatbot currently provides suggestions rather than recognizing the gender misalignment and asking a follow-up question for clarification, which is needed to proceed properly.

- **How to Retrain or Update the Model:**

Maintenance involves updating the RAG knowledge bases (policies and products) and periodically update the Vector Database:

- **Updating Policy/FAQ Documents:**

- If changes are made to the policy document, delete all existing embeddings generated for that document from the vector database (delete the collection).
- Re-embed the updated document using the configured BGE model and store the new embeddings.

- **Updating the Product Catalog:**

- New Product/Edited Info: If a product's information is edited, delete the old embedding of that product and re-embed it. If a new product is added, generate and store its embedding.
- Removed Product: If a product is removed from the catalog, delete the corresponding embedding from the Vector DB.

- **Updating LLM Prompts/Hyperparameters:**

- The core Product RAG relies on specific, manually tuned hyperparameters (Combination 3): Temperature 0.5 and Top-K 12, utilizing the refine_prompt_balanced and rerank_prompt_balanced prompts. Any changes to these prompts or values require re-evaluation using the testing methodology (manual scoring on a validation set).

- **Contacts / maintainers:**

- (Madhav) 21f1004828@ds.study.iitm.ac.in
- (Manish) 21f1005330@ds.study.iitm.ac.in
- (Neeraj) 21f3002261@ds.study.iitm.ac.in
- (Harsha) 23f2005056@ds.study.iitm.ac.in
- (Adarsh) 21f2000283@ds.study.iitm.ac.in